

RELATÓRIO TÉCNICO

Disciplina: DIM0501 — Boas Práticas de Programação

Tema: Identificação, classificação e priorização de dívida técnica

Semestre: 2026.1

1. Identificação do Grupo

Integrante 1: Daniel Vítor de Oliveira Bezerra

Linguagem utilizada: Python

Link do repositório Git: <https://github.com/danielvitorb/divida-tecnica>

2. Descrição do Sistema

- O que o sistema faz?

Permite cadastrar produtos, registrar vendas (reduzindo o estoque), listar o inventário e exibir alertas de estoque baixo. Possui também uma área restrita para que administradores acessem relatórios.

- Qual era o objetivo do código original?

Ser uma solução rápida e funcional, automatizando os processos manuais da loja.

- Em que contexto ele poderia ser utilizado?

Ideal para pequenos comércios locais, como quiosques ou cantinas escolares, onde existe apenas um terminal de vendas (computador único) operado pelo próprio dono ou por um único caixa, e que não exige acesso simultâneo em rede ou bancos de dados complexos na nuvem.

3. Inventário e Classificação da Dívida Técnica

Liste todos os itens de dívida encontrados, classificando cada um. Atribua um identificador (D1, D2, ...) que será reutilizado nas seções seguintes. Para o tipo, use as categorias da Etapa 1; para impacto, esforço e custo, use a escala de 1 a 5.

ID	Localização	Descrição do problema	Tipo de dívida	Esforço	Impacto	Custo
D 1	Função add()	Nomes de variáveis confusos (n, p, q).	Código	1	4	4
D 2	Função vender()	Permite vender quantidades negativas	Robustez	1	5	5
D 3	vender() e calcular_total()	Números mágicos e inconsistentes para regras de desconto	Código	2	4	3

D 4	Linha 7 (SENHA_AD MIN)	Senha de administrador ("1234") fixada diretamente no código-fonte (hardcoded).	Configuraç ão	1	4	3
D 5	Todo o projeto	Ausência de testes automatizados para validar cálculos financeiros e regras de estoque.	Testes	5	5	5
D 6	Todo o projeto	Uso de variável global (produtos = []) e mistura de lógicas de negócio com a UI (prints e inputs dentro das mesmas lógicas).	Arquitetura /Design	4	3	4
D 7	exportar() e relatorio_vend as()	Código morto comentado (exportar) e um comentário # TODO pendente sem implementação real.	Marcadore s	1	1	1

4. Matriz de Priorização

Posicionamento dos itens catalogados na matriz Impacto × Esforço:

	Esforço baixo	Esforço alto
Impacto alto	Ganhos rápidos (prioridade máxima) D2, D1, D4	Grandes apostas (planejar) D5, D3
Impacto baixo	Tarefas de preenchimento D7	Adiar ou aceitar D6

Ordem de prioridade resultante:

Critério usado: Razão matemática Prioridade = Impacto / Esforço (buscando o maior retorno sobre o tempo investido), com desempate analisando o custo/risco.

1. **1º D2 (Robustez em vender):** Razão 5.0 (Impacto 5 / Esforço 1). Um esforço de duas linhas evita que o caixa corrompa o estoque inteiro digitando valores negativos por engano.
2. **2º D1 (Bug em add):** Razão 4.0 (Impacto 4 / Esforço 1). Renomear variáveis e usar None resolve um problema de vazamento/estado compartilhado em segundos.
3. **3º D4 (Senha Hardcoded):** Razão 4.0 (Impacto 4 / Esforço 1). Um ajuste rápido para não deixar a senha do dono exposta no código.
4. **4º D3 (Números Mágicos):** Razão 2.0 (Impacto 4 / Esforço 2). Centralizar constantes impede descontos confusos ou errados no caixa.
5. **5º D5 (Testes Automatizados):** Razão 1.0 (Impacto 5 / Esforço 5). É fundamental, mas custa tempo.
6. **6º D7 (Marcadores):** Razão 1.0 (Impacto 1 / Esforço 1). Apagar linhas não impacta o sistema.
7. **7º D6 (Estado global/Arquitetura):** Razão 0.75 (Impacto 3 / Esforço 4). Exigiria reescrever toda a aplicação para Orientação a Objetos.

5. Roadmap de Quitação

Apresente o plano de quitação, justificando cada decisão:

- **Pagar agora:** Itens **D2** (Validação negativa) e **D1** (Bug da lista mutável em add). Estes são "ganhos rápidos". Precisam ser pagos hoje pois afetam diretamente a integridade financeira e de estoque do sistema, mas custam apenas poucas linhas de correção.
- **Pagar depois:** Itens **D3**, **D4** e **D5**. Extrair regras de desconto, ocultar a senha e criar testes de unidade são etapas essenciais que devem entrar na próxima *sprint*, pois garantirão estabilidade para que a loja evolua (ex: emissão de notas).
- **Aceitar conscientemente:** Item **D6** (Uso de variável global e ausência de modularização) e **D7** (código morto). Como o sistema roda em um ambiente pequeno (terminal isolado), o custo de refatorar tudo para Orientação a Objetos é muito alto e não compensa o esforço no momento atual, assumindo o risco técnico dessa escolha.

6. Itens Quitados (Antes vs Depois)

Apresente de 1 a 2 itens efetivamente quitados (os de maior prioridade).

Item 1 — (ID: D2)

Antes:

```
def vender(nome, quantidade):
    for i in range(len(produtos)):
        if produtos[i]["nome"] == nome:
            if produtos[i]["qtd"] >= quantidade:
                produtos[i]["qtd"] = produtos[i]["qtd"] - quantidade
                total = produtos[i]["preco"] * quantidade
                # desconto pra compras grandes
                if total > 100:
                    total = total - total * 0.1
                print("Venda realizada. Total: " + str(total))
                return total
            else:
                print("Estoque insuficiente")
                return 0
    print("Produto nao encontrado")
    return 0
```

Depois:

```
def vender(nome, quantidade):
    if quantidade <= 0:
        print("Erro: A quantidade de venda deve ser maior que zero.")
        return 0

    for i in range(len(produtos)):
        if produtos[i]["nome"] == nome:
            if produtos[i]["qtd"] < quantidade:
                print("Estoque insuficiente")
                return 0

            produtos[i]["qtd"] -= quantidade
            total = produtos[i]["preco"] * quantidade

            # desconto pra compras grandes
            if total > 100:
                total -= total * 0.1

            print("Venda realizada. Total: " + str(total))
            return total

    print("Produto nao encontrado")
    return 0
```

Explicação: O código original não validava entradas, permitindo a inclusão de números negativos e consequentemente somando estoque de forma fraudulenta. Resolvi isso aplicando o padrão Guard Clauses no início da função. Inverti o if de quantidade para retornar "Estoque insuficiente" imediatamente, eliminando o else e reduzindo o nível de indentação, deixando a lógica de desconto (comportamento) inalterada e protegida.

Item 2 — (ID: D1)

Antes:

```
# funcao que adiciona produto
def add(n, p, q, hist=[]):
    produtos.append({"nome": n, "preco": p, "qtd": q})
    hist.append(n)
    print("Produto adicionado!")
```

Depois:

```
# funcao que adiciona produto
def adicionar_produto(nome, preco, quantidade, historico=None):
    if historico is None:
        historico = []

    produtos.append({"nome": nome, "preco": preco, "qtd": quantidade})
    historico.append(nome)
    print("Produto adicionado!")
```

Explicação: Identifiquei duas dívidas graves neste trecho. Primeiro, o uso de um argumento padrão mutável (`hist=[]`), que em Python causa o reaproveitamento da mesma lista em múltiplas chamadas da função, um bug clássico e silencioso. Resolvemos utilizando a sentinela `None` e instanciando a lista localmente. Segundo, renomeamos a função e os parâmetros (`n, p, q`) para nomes expressivos (`nome, preco, quantidade`), melhorando drasticamente a legibilidade sem quebrar o uso do sistema.

7. Conclusão

Neste trabalho, aprendi que gerenciar a dívida técnica não se resume a "escrever código bonito", mas sim em fazer uma gestão ativa de risco. Entendi na prática o conceito da metáfora financeira: as dívidas escolhidas para a refatoração (como o bug da lista e a ausência de *Guard Clauses*) seriam "juros" caríssimos caso a loja enfrentasse perdas no estoque ou falhas de memória em produção.

A maior dificuldade foi na etapa de priorização (Matriz), especificamente em aceitar que uma falha de arquitetura óbvia (como as variáveis globais e mistura de lógica no menu) poderia ficar no quadrante de "Aceitar/Adiar", pois o seu esforço de refatoração para POO bloquearia muito tempo da equipe. Com isso vi que qualidade de código está diretamente ligada à entrega de valor, resolvendo primeiro o que é mais barato e protegendo mais as regras de negócio.